IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 15 November 2024, accepted 3 December 2024, date of publication 12 December 2024,
date of current version 20 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3516362

RESEARCH ARTICLE

Dynamic Load Balancing for Enhanced Network Performance in IoT-Enabled Smart Healthcare With Fog Computing

MOHAMMED ALAA ALA'ANZY[1], (Member, IEEE), RAIYMBEK ZHANUZAK[1], RAMIS AKHMEDOV[1], NADER MOHAMED[2], (Member, IEEE), AND JAMEELA AL-JAROODI[3], (Member, IEEE)

[1]Department of Computer Science, SDU University, Almaty 040900, Kazakhstan
[2]Department of Computing and Engineering Technology, Pennsylvania Western University, California, PA 15419, USA
[3]Department of Engineering, Robert Morris University, Pittsburgh, PA 15108, USA

Corresponding authors: Mohammed Alaa Ala'anzy (m.alanzy@ieee.org) and Raiymbek Zhanuzak (raiymbekzhanuzak@gmail.com)

This work was supported in part by the Department of Engineering, Robert Morris University, Pittsburgh, USA; and in part by the Science Department, SDU University, Kazakhstan.

ABSTRACT The rapid expansion of Internet of Things (IoT) devices in healthcare has increased data volumes, creating challenges for the efficiency and latency of real-time monitoring systems. Traditional cloud computing often encounters high latency and network congestion, making it unsuitable for time-sensitive healthcare applications. Although fog computing introduces intermediary nodes to mitigate these issues, existing approaches frequently lack efficient workload distribution, leading to performance bottlenecks. To address these limitations, an Optimised Load Balancing (OLB) algorithm is proposed, to allocate workloads effectively across fog nodes and to reduce communication and computational delays. The system follows a three-tier architecture: 1) Sensor data collection, where patient-worn sensors transmit vital signs; 2) a Fog layer for real-time analysis near base stations; and 3) Cloud storage and user access via mobile devices. Simulations conducted using the iFogSim toolkit demonstrate that OLB achieves a 28% reduction in latency, a 15% improvement in network usage, a 20% reduction in execution time, a 25% decrease in energy consumption, and a 22% reduction in execution cost compared to existing methods, including the Load Balancing Scheme (LBS), Fog Node Placement Algorithm (FNPA), Load-Aware Balancing (LAB) scheme, and Mobile Edge Computing (MEC). By dynamically adjusting workload distribution based on real-time traffic and computational capacity, the proposed fog-based solution provides a responsive, energy-efficient, and cost-effective approach to healthcare data management, surpassing MEC and other state-of-the-art algorithms in adaptability and resource efficiency.

INDEX TERMS Fog computing, health monitoring, IoT, latency, load balancing, network usage.

I. INTRODUCTION

The widespread adoption of the IoT has permeated daily life [1], impacting areas from smart cities and home automation to healthcare. In healthcare, IoT has transformed workflows by automating tasks previously done manually, enabling continuous health monitoring of patients. Real-time data processing is paramount in healthcare environments, especially in patient monitoring, where delays in processing can have life-threatening consequences. Despite these advancements, IoT healthcare systems still face critical challenges in managing high data volumes, reducing latency, and addressing network congestion to enable timely and effective interventions.

Researchers have proposed various architectural frameworks that combine IoT and cloud computing, yet many

The associate editor coordinating the review of this manuscript and approving it for publication was Jad Nasreddine.

struggle to meet the stringent requirements of real-time healthcare applications, leaving room for improvement in both processing speed and reliability.

Cloud computing has become a widely adopted solution for data processing and storage in healthcare applications. However, it faces challenges such as delays in data transmission, managing large volumes of data, and network congestion, often stemming from the geographical distance between cloud servers and IoT devices [2]. To address these limitations, fog computing has emerged as a new paradigm. Fog nodes are strategically positioned computing devices that connect devices within a network, providing locally hosted computing and storage resources [3]. This approach helps to alleviate some of the burdens on cloud computing by bringing processing and storage closer to the data source.

This geographically distributed framework offers a more flexible approach to managing healthcare data than traditional cloud computing. Figure 1 illustrates the structure of Fog Computing, highlighting the interactions between the fog and cloud layers. The fog layer acts as an intermediary, processing data close to IoT devices to reduce latency and enhance security, while the cloud layer provides scalable computational power and long-term data storage. Together, these layers facilitate efficient task distribution and dynamic data management, allowing healthcare systems to respond promptly to real-time demands while ensuring data integrity and availability.

The healthcare industry faces an ever-increasing demand for real-time data processing, driven by the proliferation of IoT-enabled health monitoring devices [4]. These systems are critical for managing chronic conditions, responding to emergencies, and providing timely care. However, traditional cloud-based architectures are often inadequate for such applications due to inherent limitations such as high latency, network congestion, and delays in communication caused by the geographical distance between IoT devices and centralised servers. These challenges highlight the pressing need for solutions that can manage large-scale medical data processing in real time.

In order to overcome these limitations, fog computing utilises strategically positioned computing devices called fog nodes, which form the core of this architecture. These fog nodes connect various devices within the network and provide localised computing and storage resources, creating a geographically distributed framework. Compared to traditional cloud computing, fog computing offers a more adaptable and secure approach to managing healthcare data while minimising bandwidth consumption [3].

Several approaches have been explored to mitigate these issues, including cloud computing [5], [6] and basic fog computing architectures [7]. However, current research has primarily focused on integrating fog computing with IoT healthcare systems without addressing the optimal placement and efficient load balancing of workloads across fog nodes. Many existing systems lack the mechanisms to dynamically allocate resources, leading to inefficiencies in handling critical data. This creates a need for research on managing the distribution of healthcare data and workloads across fog networks, particularly in large, geographically dispersed environments.

To meet this need, an OLB algorithm is proposed and designed specifically for IoT-enabled healthcare systems. By intelligently distributing workloads between fog nodes based on traffic and computational demands, OLB reduces both latency and network usage, ensuring the efficient and timely processing of healthcare data. Through extensive simulations using the iFogSim toolkit, the OLB algorithm outperforms existing load-balancing techniques in terms of both latency reduction and network efficiency. The key contributions of this research include the novel fog-based architecture and a comprehensive evaluation of its performance in real-time healthcare environments.

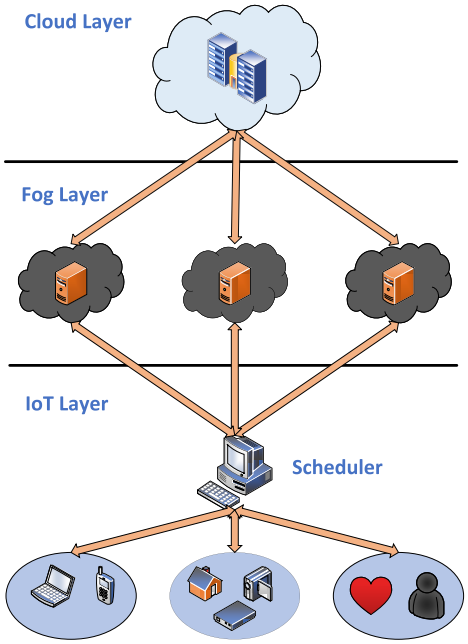


FIGURE 1. Architecture of fog computing.

Healthcare systems across the globe are currently facing significant challenges, exacerbated by factors such as an ageing population [8], increasing rates of chronic diseases, and a shortage of healthcare professionals [9]. According to a recent report by the World Health Organisation (WHO), the global population of individuals aged 60 and older is projected to reach 2 billion by 2050, compared to 900 million in 2015 [10]. Chronic illnesses are responsible for 60% of global deaths and a staggering 85% of fatalities in China alone. In the United States, chronic diseases contribute approximately 75% of total healthcare costs [11]. The lack of adequate medical resources, especially in remote areas

with growing patient populations, further compounds these challenges [9].

The proposed fog-based architecture transmits patient health status data by processing data collected on and transmitted by wearable sensors. It is made up of three levels as follows.

- Sensor Data Collection: In this first tier, patients wear sensors that measure and transmit vital signs such as body temperature, heart rate, pulse oximetry, etc.
- Fog Layer for Analysis: The middle tier consists of the fog layer, with fog nodes strategically placed near Base Stations (BS) where all IoT devices are connected. Sensor data from Tier 1 is sent to fog nodes, which analyse the data in real-time to determine if immediate medical attention is needed for a patient.
- Cloud Storage and User Interface: The results are then sent to the cloud for storage through a top-tier proxy server. Furthermore, fog nodes send health status updates to the smartphones of the patients for convenient access.

The architecture is designed to offer continuous, real-time medical assistance to patients by utilising fog computing for time-sensitive applications. The primary benefit of Fog computing lies in its ability to bring processing resources closer to users, such as patients, which allows for efficient management of substantial amounts of sensor data. As a result, the proposed system integrates fog computing to meet the crucial requirements of real-time and effective healthcare data processing.

Recent studies have highlighted that data traffic and processing delays present significant challenges for IoT data streams [12]. The proposed OLB algorithm addresses this issue by enabling IoT devices to select the most suitable fog nodes for data processing, thereby reducing latency. To evaluate its efficiency, two sets of simulations were conducted using the iFogSim toolkit. The first scenario compared the OLB algorithm with LBS, FNPA, and LAB schemes, focusing on latency and network usage. The results demonstrated the superior performance of the OLB algorithm in reducing latency and optimising network resources. The second scenario assessed the OLB algorithm against the MEC algorithm across broader metrics, including execution time, energy consumption, and execution cost, where the OLB algorithm consistently outperformed MEC by achieving lower latency, improved network efficiency, and reduced energy consumption.

The contributions of this paper can therefore be summarised as follows:

1) A three-tiered health monitoring architecture based on fog computing is proposed, featuring fog nodes in the middle layer. Patient sensor data is streamed from smartphones to these fog nodes, where it is processed to evaluate health status. The processed results are then sent back to the smartphone and subsequently uploaded to the cloud for long-term storage and analysis.
2) The OLB algorithm is developed to intelligently distribute workloads across fog nodes in large health monitoring networks. It optimises both traffic and computational loads in real-time, enhancing the responsiveness and efficiency of the system.
3) The OLB algorithm is designed to dynamically adjust resource allocation based on traffic loads and computational capacity at each fog node. This dual consideration effectively reduces both communication and computation latencies, making it particularly suitable for real-time healthcare applications.

The organisation of this paper is as follows: Section II reviews contemporary research on health monitoring architectures and load balancing within fog-based systems. Section III introduces the proposed architecture for health monitoring systems, including the proposed algorithm. Section IV outlines the performance metrics, simulation toolkit, experimental setup, and results. Finally, Section V concludes the paper and suggests directions for future research.

II. RELATED WORK

This section is a review of the existing literature on fog computing and its application within the IoT ecosystem, particularly in the healthcare sector [13]. There is a rising trend for research on fog computing efficiently that manages healthcare data within the IoT ecosystem. Certain data from IoT devices is deemed critical; for example, heart patients are managed at fog nodes or edge devices in fog computing, which possess high processing capabilities. This proximity to IoT devices (fog nodes) significantly reduces delay, response time, and latency compared to cloud computing.

The authors in [14] presented an RBAC-based load balancing and resource management framework for IoT-edgefog networks called RBAC-LBRM. This framework aims to resolve network load imbalances and unauthorised access, which can impair performance. By offloading tasks to edge and fog layers, RBAC-LBRM enhances CPU usage, memory utilisation, delay, and jitter compared to other methods. While it improves resource utilisation and access control in dynamic IoT environments, RBAC-LBRM lacks real-time adaptability in task allocation, which is vital for applications such as healthcare. It does not provide a comprehensive strategy for optimising both latency and network performance, focusing on load balancing and access control. This gap emphasises the significance of the OLB algorithm, which dynamically manages traffic and computing loads to improve both latency and network performance, addressing the limitations of existing frameworks like RBAC-LBRM.

Furthermore, the authors in [15] introduced the Gateway Validation Module Placement (GVMP) strategy to address application module placement in IoT-fog-cloud environments. The GVMP strategy is designed to reduce average application latency and network usage by strategically placing modules on heterogeneous fog devices near the network

edge. The authors argue that relying solely on cloud-based servers in IoT applications can result in high latency and network congestion, especially in smart city scenarios where data volumes are large. By validating gateway capabilities, the GVMP strategy effectively selects optimal fog nodes for task execution, achieving notable improvements over the Edgeward Module Placement (EWMP) strategy. The GVMP strategy demonstrates clear advantages in optimising latency and network usage; however, its approach does not explicitly address the adaptive allocation needs in highly dynamic environments. IoT networks often face fluctuating demands and real-time workload shifts, and the lack of a dynamic adjustment mechanism may limit the responsiveness of the GVMP in large-scale deployments. Additionally, it needs to be stressed that the GVMP technique is only compared with the EWMP strategy here. The relative performance of GVMP against other adaptive load-balancing and multi-tiered algorithms is left unexplored. This limits a broader understanding of the efficiency of GVMP in more complex, latency-sensitive applications.

Next, [16] introduced HealthFog, a fog-based smart system to diagnose heart disease. Their solution integrates software and hardware to ensure fast and reliable data transmission. Another study ([17]) presented a Fog computing-based patient monitoring system for ambient assisted living (FAAL), a fog-based framework for monitoring patients with chronic neurological conditions. This system uses a clustering algorithm in the fog layer to reduce the processing burden on individual fog nodes. Next, the researchers in [18] proposed a similar fog-based eHealth application that collects physiological data and environmental factors such as light and air quality. In [19] the authors presented a three-tier architecture for the Internet of Health Things (IoHT) that considers user mobility. Their system uses sensor nodes for data collection, fog nodes for managing health parameters, and a cloud server for processing during emergencies.

Even though the studies referenced under [16], [17], [18], [19] evaluated fog-based health monitoring systems in comparison to cloud-based solutions. However, the proposed OLB algorithm adopts a different methodology. It compares the fog-enabled system with location-based services and two other fog-based implementations: FNPA and LAB.

To begin with, [20] proposes an e-health system for monitoring elderly health based on IoT and Fog computing. The system uses the Mysignals HW V2 platform and an Android app to collect physiological and general health parameters periodically, allowing elderly individuals and their families to track health status and communicate with healthcare providers. The evaluation results indicate that users find the system useful, easy to use, and easy to learn, suggesting it can enhance the quality of healthcare for the elderly. In the system under [20], a fog layer analyses sensor data and sends it to the cloud via a specific data format (JSON) and communication method (REST API).

Elsewhere, [21] describes a five-tier architecture aimed at managing the real-time streaming and processing of data from healthcare sensors and IoT devices at the edge, thus addressing latency and bandwidth challenges inherent in cloud processing. The [21] architecture incorporates IoT, edge/fog computing, data integration, cloud computing, and data analytics to enable real-time event detection, alert notifications, and monitoring dashboards, with key components including Apache Kafka for distributed messaging and Apache Storm for stream processing. However, their proposed systems were not evaluated through simulations.

In [22] and [23] fog-based systems were proposed for monitoring elderly and unattended individuals. Also, [24] presented a fog-enabled system for monitoring blood sugar levels, using a specific decision-making tool to predict diabetes risk [24]. While these studies used simulations to assess their systems [22], [23], [24], they lacked detailed results on performance measures such as latency and network usage.

This paper stands out by comparing a proposed fog-based health monitoring system with three established fog-based approaches as well as the MEC framework, offering comprehensive simulation results across key performance metrics, including latency, network usage, execution time, and energy consumption. This multi-faceted evaluation demonstrates the advantages of the proposed approach, particularly its responsiveness to real-time demands and efficiency in resource utilisation.

Different work reported in [25] presented a fog-based health monitoring system specifically designed for diabetic patients with cardiovascular conditions. Other researchers in [26] developed the COVID-SAFE framework, a fog-based system that aims to minimise the risk of exposure to COVID-19. This framework leverages fog nodes equipped with Machine Learning (ML) tools to process and analyse data. Furthermore, [27] introduced a fog-based system that enables home hospitalisation, allowing patients to receive care at home while doctors remotely monitor their health and surroundings. However, none of the studies mentioned in [25], [26], and [27] evaluated their proposed systems in terms of latency and network usage, which are crucial performance metrics for real-time healthcare applications.

In [28], researchers proposed an IoT-based healthcare system that leverages fog nodes in various locations to manage patient requests across different cities. This system monitors patient health data on fog nodes, with critical cases being immediately forwarded to the cloud server for further attention. Non-critical situations are managed by the fog nodes themselves. Although [28] evaluated different system layouts through simulations to assess performance, their work lacked comparisons with existing cloud or fog-based healthcare systems.

Next, [29] presented a novel task scheduling algorithm for n-tier fog computing environments, focusing on energy efficiency and fault tolerance in IoT applications. This

approach addresses the need for real-time processing with minimal latency and network usage by rescheduling tasks to alternative fog nodes in case of device failure. The algorithm, based on modified particle swarm optimisation, incorporates reactive fault tolerance, allowing for rescheduling without significant delays. The authors demonstrate the effectiveness of the algorithm by achieving reductions in energy consumption, latency, and network bandwidth utilisation, along with improved system reliability and task success rate. While the proposed algorithm offers valuable improvements in reliability and energy efficiency, its scalability in larger, more complex fog computing networks remains unaddressed. Additionally, the paper does not compare its approach with other fault-tolerant or energy-efficient algorithms, limiting the positioning framework within existing literature. As the algorithm relied on a modified particle swarm optimisation, it may also introduce computational overheads that could affect performance in real-time applications with more extensive datasets or device networks.

Other work in [30] presented a fog-enabled health monitoring architecture where fog nodes process data using a specially-designed algorithm for scheduling tasks efficiently. Also, different studies presented a combined cloud-fog architecture for patient health monitoring, employing a unique strategy for allocating tasks [31]. Both [30] and [31] evaluated their proposed architectures and algorithms through simulations. However, their comparisons were limited to cloud-only systems, and neither evaluated their models against other fog-based implementations.

This paper compares the proposed fog-based health monitoring system with LBS, FNPA, LAB, and the MEC framework. This comprehensive comparison provides a more detailed assessment of the effectiveness of the system across multiple performance metrics, highlighting its advantages in latency and network usage relative to existing approaches.

A three-tiered architecture for remote pain monitoring was proposed by authors in [34]. Their system uses fog nodes equipped with signal-processing techniques to detect pain. While their approach demonstrated lower latency compared to cloud-based solutions, it faced scalability limitations. As the number of patients increased, a single fog node managing all hospital data became overwhelmed.

Next, [35] proposed the DynaFog framework to address task offloading in dynamic fog computing networks with multi-hop IoT access points (APs) that connect IoT devices to fog nodes. The DynaFog framework encompasses key components such as skill-based fog node selection, optimal path routing, and strategic decision-making for local versus remote task computation. The task offloading problem is formulated as an integer linear program (ILP). Given the complexity of the solution space, a heuristic strategy driven by a greedy approach is applied to improve computational efficiency. The DynaFog solution considers multi-hop routing, energy usage, delay, and network dynamics, offering a comprehensive approach to resource optimisation in IoTfog networks. The emphasis of the framework on multi-hop communication and task offloading efficiency distinguishes it from simpler fog solutions. While DynaFog demonstrates significant reductions in delay and energy consumption, a weakness lies in its use of a heuristic approach, which may limit solution optimality in highly dynamic or large-scale fog networks. Additionally, while the authors focus on delay and energy metrics, further evaluation against other real-time load-balancing and resource management frameworks would be valuable to confirm its effectiveness in comparison to state-of-the-art adaptive methods.

Another study by [38] addressed scalability challenges by introducing a load-balancing algorithm that distributes tasks across all three layers of their architecture. A threshold is set for the fog layer to manage workload. When this threshold is exceeded, jobs are automatically forwarded to the cloud server in the top layer. Simulations showed that this approach reduced average processing time.

Both [32] and [33] have proposed various resource allocation frameworks to address the challenges of processing data in IoT environments using fog and cloud computing paradigms. Furthermore, in [33], the authors focused on the dynamic and complex nature of fog computing in IoT transportation, emphasising the need for reliable resource allocation and management to accommodate changing user demands and the inherent autonomy and wireless communication constraints of fog devices. This strategy, tested using the iFogSim2 simulation tool, demonstrated a significant reduction in latency and energy consumption. Meanwhile, the work in [32] highlights the limitations of cloud computing for delay-sensitive IoT applications due to high transmission latency and energy consumption, advocating for a collaborative approach between fog and cloud paradigms. The authors of [32] propose a two-stage resource allocation framework that classifies and allocates tasks based on their requirements, using a Bayes classifier and Crayfish optimisation algorithm to optimise resource use and minimise delay and execution time. However, neither [33] nor [32] consider the impact of network usage on resource allocation, an area which this research aims to address by integrating network considerations into the resource allocation process. By doing this, overall system efficiency and reliability are enhanced.

Several other studies have explored different load-balancing techniques in fog computing environments. For instance, [39] introduced the Multi-tenant Load Distribution Algorithm for Fog Environments (MtLDF), demonstrating its effectiveness compared to the Delay-Driven Load Distribution (DDLD) approach. Different work in [40] presented the Dynamic Resource Allocation Method (DRAM) for load balancing in fog-based systems. This method uses a combination of static resource allocation and dynamic service migration to optimise resource utilisation, although it did not evaluate latency. Next, the strategy proposed in [41] focused on energy efficiency by assigning tasks to fog nodes based on

TABLE 1. Summary of the influential contributions of the most related studies.

Ref.	Multi-tier	FC	Real-time	Scalable	Dynamic	Latency	Network Usage	Exec. Time	Cost	EC
[14]	✓	✓	✓	✓	✗	✓	✓	✗	✗	✗
[15]	✓	✓	✗	✗	✗	✓	✓	✗	✗	✗
[19]	✓	✓	✓	✗	✓	✓	✗	✓	✗	✓
[29]	✓	✗	✓	✗	✓	✓	✓	✗	✗	✓
[32]	✓	✓	✓	✓	✗	✓	✗	✓	✗	✗
[33]	✓	✓	✓	✗	✓	✓	✗	✗	✓	✓
[34]	✓	✓	✓	✗	✗	✓	✓	✗	✗	✗
[35]	✓	✗	✗	✗	✓	✓	✗	✗	✗	✓
LBS [36]	✓	✓	✗	✓	✓	✓	✓	✗	✗	✗
MEC [37]	✓	✓	✗	✗	✓	✓	✓	✓	✓	✓
Proposed	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

FC = Fog Comparison; Exec. Time = Execution Time; EC = Energy Consumption

their processing power and energy consumption. Simulations showed that this approach significantly improved application latency, network usage, and energy consumption.

Building on these existing approaches, the proposed OLB algorithm is designed to further reduce latency and network utilisation within a big data health monitoring environment. By dynamically allocating workloads based on real-time traffic and computational capacity, OLB enhances responsiveness and optimises resource usage, making it particularly suited to the demands of large-scale, time-sensitive healthcare applications.

FNPA, introduced in [42], optimises resource utilisation by associating IoT devices with the nearest suitable fog nodes with sufficient CPU, RAM and bandwidth. FNPA demonstrated efficiency over cloud only implementation and a fog-node approach with minimum distance, showing a significant reduction in latency, execution costs and network usage.

Next, two types of latency were identified by [43] in IoT data flows, namely network delay (due to volume of service requests) and computing delay (due to allocation of resources for service requests).

Another team in [31] proposed a Workload Allocation (WALL) scheme and an application-aware workload allocation (AREA) scheme for hierarchical cloud computing (HCC). These schemes allocate IoT user requests to suitable cloudlets, reducing network and computing delay, and resulting in an average response time reduction.

LAB was introduced in [43] to mitigate the latency issues in IoT data flow. The LAB scheme effectively connects user devices to appropriate base stations, enhancing the overall efficiency and performance of IoT networks.

Many previous papers such as [19], [20], [21], [22], [25], [26], [27], [30], [31], and [41] have only compared their suggested fog-based solutions to cloud-based solutions, disregarding performance metrics such as latency and network utilisation. Some researchers focus solely on one metric; for example, in [40], the impact on latency is not discussed, while in [32], [33], [38], [39], and [44], network usage is not taken into account when evaluating performance. Additionally, some prior works such as [20] and [21] did not perform experiments to evaluate their approaches.

In this study, extensive simulations were conducted across five distinct topological configurations, comparing the proposed algorithm with four other implementations: three fog-based approaches and the MEC framework [37]. Unlike MEC, which uses static offloading strategies to edge nodes, the OLB algorithm dynamically distributes workloads across fog nodes based on real-time traffic and node capacity. This adaptability enables the OLB algorithm to prevent node congestion and optimise resource use more effectively. Results show that the proposed algorithm consistently outperforms all configurations, achieving lower latency and better network utilisation across all evaluated scenarios.

While numerous researchers have introduced fog-based architectures for health monitoring systems, emphasising their advantages over traditional cloud-based setups, there remains a gap in comparative analyses with other fog-based solutions specifically designed for healthcare. This omission highlights the need for a fog-based approach that can demonstrably outperform existing solutions. Additionally, while latency is a critical performance metric in health monitoring systems, efficient network usage is equally important, especially in data-intensive healthcare environments. However, many studies have focused solely on latency, neglecting network usage in their evaluations. To address these gaps, it is essential to assess the proposed approach comprehensively, with a focus on both latency and network usage, thereby providing a more holistic measure of its effectiveness.

The related work discussed in this section is presented in Table 1, which highlights key aspects of various approaches in fog and IoT-based environments. Table 1 provides a comparative overview of these studies, focusing on critical features such as multi-tier architecture, scalability, latency reduction, energy efficiency, and network performance. This comparison underscores the specific challenges addressed by each approach and identifies existing gaps in dynamic load balancing and real-time adaptability, which the proposed method aims to address more comprehensively.

III. THE PROPOSED WORK

In healthcare environments, real-time data processing is paramount, particularly for applications like patient monitoring, where delays in processing could have life-threatening consequences. Existing fog-based load balancing methods, such as FNPA and LAB, tend to rely on static allocation strategies or complex formulas, which might not adequately address the dynamic workload fluctuations inherent in healthcare systems. The proposed OLB algorithm offers a flexible and responsive solution by dynamically adjusting the allocation of tasks based on real-time conditions, both in terms of traffic and computational load.

The ability of the OLB algorithm to intelligently manage traffic and computational loads in real-time not only reduces system latency but also prevents potential network overloads that could degrade the quality of healthcare services. This is particularly crucial for healthcare environments, where IoT device usage and data flows can change rapidly, especially during emergencies or spikes in monitoring activity.

A. SYSTEM ARCHITECTURE

This section presents a three-tier framework for a health monitoring system utilising fog computing. The first tier consists of sensors (S1, S2, S3, and S4) connected to the patient, which monitor vital signs such as body temperature, heart rate, and pulse rate. These sensors transmit data to nearby smartphones (A1 and A2), which act as intermediaries, forwarding the information to the second tier. The second tier comprises fog nodes (FogNode 1 and FogNode 2), located near BS at the network edge. These fog nodes perform real-time analysis of the data received from the sensors and deliver health status results back to the smartphones. Their strategic positioning ensures prompt responses and reduced latency, which is critical for real-time healthcare applications. The third tier involves a cloud data centre, which serves as a long-term storage facility for processed health data. Communication between the fog nodes and the cloud is facilitated by a proxy server, which acts as an intermediary. In this configuration, the cloud primarily functions as a storage centre, while fog nodes manage computational tasks closer to the data source. Figure 2 illustrates this architecture, showing the flow of data between sensors, smartphones, fog nodes, the proxy server, and the cloud. The figure captures the hierarchical structure and highlights the roles of each component in the health monitoring system.

1) **IoT Layer:** In the health monitoring system under consideration, the IoT layer consists of IoT devices. These devices, such as sensors linked to smartphones, are responsible for monitoring the body temperature, heart rate, and pulse rate of the patient. The collected data is then sent to a higher-level fog device for analysis to determine the condition of the patient.
2) **Fog Layer:** The fog layer serves as a connection between the sensor network (IoT layer) and the cloud layer. Sensor data streams are directed to fog nodes for initial processing before potentially being sent to the cloud for storage or further analysis. These fog nodes are strategically positioned within BSs. The coverage areas of these BSs may overlap, allowing each IoT device to connect to a single fog node or BS even if it is within the range of multiple stations. In comparison to the cloud, fog nodes have limitations in networking capabilities, processing power, and storage capacity. Nevertheless, within the proposed fog-based health monitoring architecture, they are crucial for processing and analysing real-time data closer to the users (patients wearing the sensors). This proximity to the data source enables quicker processing and reduces latency.
3) **Cloud Layer:** The architecture of the health management system includes a cloud server and a proxy server at the top. The cloud server offers additional computing and storage capabilities, while the proxy server acts as an intermediary for data exchange between the fog layer and the cloud server [45], [46], [47]. Once the sensor data is processed, the fog nodes transmit the health status outcomes of the patient to the cloud server, which maintains a continuous database for recording the results. Moreover, the data stored in the cloud can be accessed at any time.
4) **Application Model:** Within the system is the fog computing application model. There are multiple application modules responsible for executing specific operations on the collected data. In the proposed health monitoring system based on fog computing, the application model consists of three modules: the client module, the processing module, and the storage module.

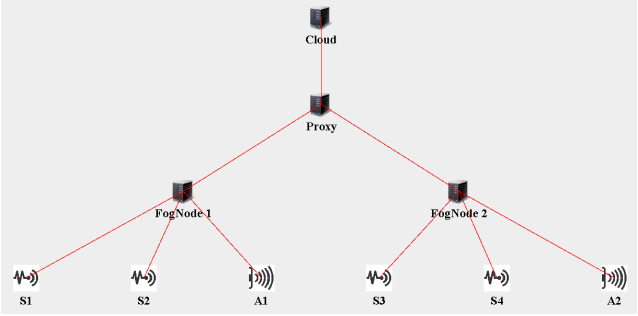


FIGURE 2. Fog computing-based topology for health monitoring system.

The individual functions as the customer for the health monitoring system, where the Client Module serves as the primary interface. In the Client Module, sensors collect information on individual health metrics, such as body temperature, heart rate, and pulse rate. Subsequently, this information is sent through the individual smartphone to the Processing Module located in the fog node for evaluation. The findings concerning the health condition for the individuals are then sent back to the client module on the smartphone for visualisation.

The Processing Module plays a critical role in analysing the incoming data. It utilises algorithms to evaluate the sensor data in real-time, comparing it against predefined thresholds and historical data patterns. If the data indicates any anomalies or critical conditions, the module triggers alerts for healthcare providers and informs the patient through the Client Module. Additionally, this module aggregates data, ensuring that comprehensive health status reports are generated for further analysis.

The Storage Module, which is responsible for maintaining records, is integrated with the cloud server. Analysed outcomes from the Processing Module are also transferred to the Storage Module, allowing for long-term data retention and enabling healthcare professionals to access historical patient data for ongoing assessments and treatment planning.

The integration of these modules allows for real-time monitoring, efficient data processing, and reliable data storage, ultimately improving patient care. The application model of the proposed health monitoring system is depicted in Figure 3.

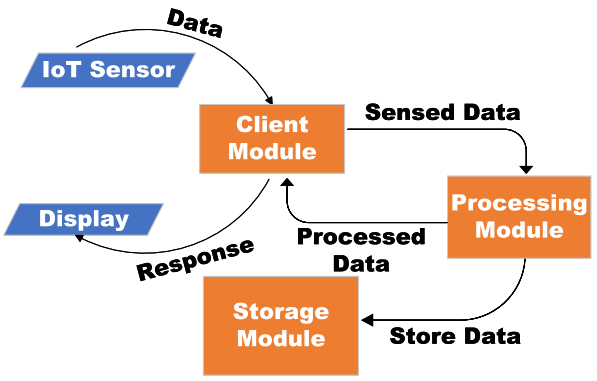


FIGURE 3. Application model.

B. OPTIMISED LOAD BALANCING ALGORITHM

In this study, fog nodes are positioned alongside BSs to facilitate cellular communication between IoT devices and the fog network. These fog nodes enable real-time processing and data transmission, significantly reducing latency, which is crucial for healthcare applications that demand fast response times. The coverage zones of neighbouring BSs may overlap, allowing IoT devices in these areas to connect with multiple BSs. However, as more devices connect, network congestion may occur, leading to increased data flow latency, which can be broken into two main categories: communication latency caused by traffic volume and computing latency, which arises from the computational load on the BS and fog nodes.

The proposed OLB algorithm introduces a novel approach by dynamically managing both traffic and computing loads. Unlike existing algorithms such as FNPA or LAB, which typically focus on either proximity or computational capacity, the OLB algorithm intelligently distributes workloads, defined here as the continuous stream of data generated by IoT devices, based on real-time traffic metrics and computing capacity. This dual consideration ensures that the system can respond quickly to fluctuations in data flow and workload, which is particularly important in healthcare applications requiring real-time monitoring. Additionally, the unique adaptability of the OLB algorithm in a three-layer architecture enables optimised task allocation across fog nodes in real-time, overcoming the limitations of static configurations. By simultaneously addressing traffic demands and computing power, the approach enhances latency, network usage, execution time, and other performance metrics, offering a more comprehensive solution compared to current methods that prioritise proximity or computational capacity independently.

In contrast to conventional load balancing techniques such as the LBS scheme presented in [36], which recalculates traffic and computing loads for all devices within the coverage area of the fog node, only OLB recalculates these metrics for the specific device that needs to be allocated. This optimisation significantly reduces redundant calculations, enhancing the overall efficiency of the fog network. Additionally, the OLB algorithm incorporates an array-based method to dynamically update traffic load density and computing load density, improving the scalability of the algorithm and performance in large healthcare deployments.

TABLE 2. Key symbols used in equations.

Symbol	Explanation
J	A collection of base stations or fog nodes
D	A collection of IoT devices
$P(x)$	Power of the IoT device at position x
$g(x)$	The IoT device is channel gain at position x
$fl(x)$	Arrival rate of flow at point x
$l(x)$	Size of traffic at site x
c_j	IoT device capacity at location x
$v(x)$	Calculating the dataflow size at position x
C_j	The fog node j computing capacity
$e_j^a(x)$	Density of IoT device traffic at location x
$e_j^b(x)$	Calculating the IoT device load density at position x
TL_j	Base station traffic load j
CL_j	Base station j computation of load
L_m	Ratio of communication latency
L_p	Calculating the latency ratio

To provide a clear understanding of the OLB algorithm, Table 2 lists the symbols used throughout the algorithm, while Table 3 outlines the specific values used in the proposed work simulations.

These tables offer key insights into the parameters involved in the load-balancing process, ensuring that both communication latency and computational latency are minimised through optimal resource allocation.

The OLB algorithm optimises both communication and computational latencies by continuously monitoring traffic conditions and the computational demands of each fog node. By doing so, it can predict when a BS or fog node is at risk of becoming overloaded and redistributing tasks before system

TABLE 3. Symbol values used in the equations.

Parameters	Values
$fl(x)$	0.50 flows/seconds
$l(x)$	0.05 MBits
$v(x)$	5000 CPU cycles
$P(x)$	100 mW
C_j	7.0 x 10^6
Uplink Frequency BW	10 MHz
Carrier frequency	2110 MHz
Noise power level	-104 dBm

Algorithm 1 Optimised Load Balancing Algorithm

```
for int i = 0; i < numberOfTasks; i++ do
    int minIndex = 0;
    for int j = 0; j < numberOfNodes; j++ do
        if fog node j has minimum total latency
        $$ device in coverage area of j then
            minIndex = j;
        end
    end
    Calculate distance;
    Calculate ea_x by using Equation (4);
    Calculate eb_x by using Equation (7);
    Calculate TL[minIndex] with Equation (12);
    Calculate CL[minIndex] with Equation (13);
    Calculate LM[minIndex] by using Equation (6);
    Calculate LP[minIndex] by using Equation (9);
    Set parent of device to FogNode[minIndex];
end
for int i = 0; i < numberOfNodes; i++ do
    latency += LM[i] + LP[i];
end
```

performance degrades. This level of real-time adaptability is a crucial improvement over static or formulaic approaches such as FNPA and LAB, which may not respond adequately to sudden spikes in demand or fluctuating network conditions.

The proposed OLB algorithm starts by preparing to assign tasks to available fog nodes, initialising a variable to track the fog node with the lowest latency. For each task, it iterates through all fog nodes to determine the one with the minimum total latency by calculating both communication and computational loads based on real-time traffic conditions. Once the fog node with the least latency is identified, the algorithm updates its load by calculating traffic density and computing density, adjusting the traffic load and computational load arrays to reflect this assignment. After assigning each task, the algorithm then accumulates the communication and computing latency across all fog nodes, maintaining a dynamically optimised allocation of tasks. This process ensures efficient task distribution, minimising redundant calculations and adapting to real-time changes in workload and resource availability.

The following pseudo-code outlines the key steps of the OLB algorithm, which calculates the traffic load and computational load density for each fog node and updates these values as new devices are connected. This is represented in Algorithm 1, with the steps clearly outlined to ensure efficient load balancing. As illustrated in Figure 4, the flowchart outlines the sequential decision-making process of the OLB algorithm, showing how IoT devices are assigned to fog nodes based on real-time traffic and resource metrics. The figure provides a step-by-step visualisation of how the algorithm functions in practice, offering a clear view of the optimisation process. Once an IoT device is assigned to a fog node, the traffic and computational loads are updated dynamically in traffic load density and computing load density arrays, reducing the need for recalculations for already assigned devices. This efficient handling of load allocation ensures that the performance of each fog node is maximised, maintaining system stability even as the number of connected devices increases.

To analyse the performance of the proposed algorithm, several foundational equations and formulas are helpful. For an IoT device operating within area A at a given location x, key parameters include transmission power $P(x)$, channel gain $g(x)$, and noise power σ^2. The Signal-to-Noise Ratio (SNR) at location x is calculated using Equation (1):

$$SNR(x) = \frac{P(x) \times g(x)}{\sigma^2}, \tag{1}$$

where the channel gain $g(x)$ for an IoT device located at x is computed using Equation (2). This formula helps quantify how well the signal propagates based on the wavelength λ and the distance d between the device and the BS.

$$g(x) = 10 \log_{10} \left[\frac{\lambda^2}{(4\pi d)^2} \right]. \tag{2}$$

The wavelength λ can be determined by dividing the speed of light by the carrier frequency of the BS. Using Equation (3), the capacity of an IoT device connected to BS j with

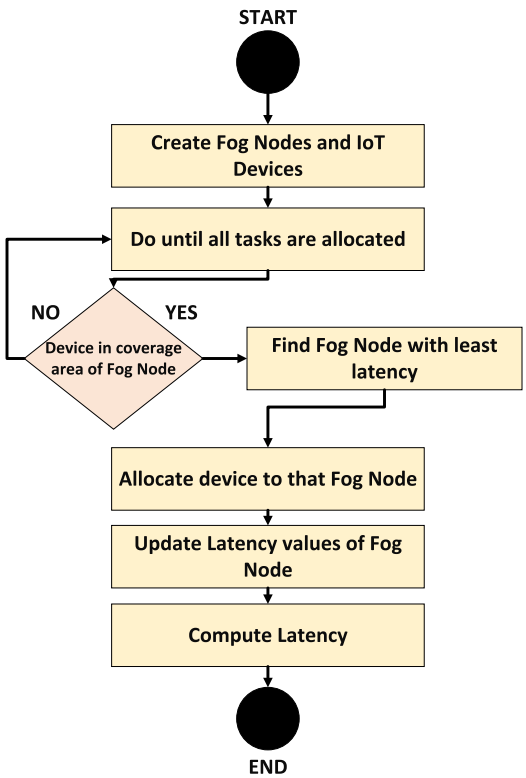


FIGURE 4. The algorithm flowchart.

bandwidth BW_j is calculated based on the SNR of the device at location x,

$$c_j(x) = BW_j \times (1 + \text{SNR}(x)). \tag{3}$$

The traffic load density for the j-th BS, corresponding to the IoT device at location x, is expressed using Equation (4). This helps in estimating the load generated by the device relative to its capacity,

$$e_j^a(x) = \frac{fl(x) \times l(x)}{c_j(x)}. \tag{4}$$

According to [44], IoT data flows have an average flow rate denoted by $f_l(x)$, and $l(x)$ represents the size of the flow traffic. These flows follow the poisson point process. The overall traffic load on a base station can be computed by summing up the traffic load densities for all devices in the area A, as described in Equation (5),

$$TL_j = \sum_{x \in A} e_j^a(x). \tag{5}$$

The communication latency ratio L_m for BS j can be derived using Equation (6), which depends on the total traffic load of the base station, TL_j, as modelled in [44],

$$L_{m(j)} = \frac{TL_j}{1 - TL_j}. \tag{6}$$

The computing load density for an IoT device at location x depends on the average flow size $v(x)$ of the device. Equation (7) is used to calculate the computing load density for fog nodes, which takes into account the processing power of the corresponding fog node C_j:

$$e_j^b(x) = \frac{f_l(x) \times v(x)}{C_j}. \tag{7}$$

The total computing load for BS j can be determined by summing the computing load densities for all IoT devices associated with it. Equation (8) shows how this is calculated:

$$CL_j = \sum_{x \in A} e_j^b(x). \tag{8}$$

Finally, the computing latency ratio L_p for BS j can be calculated using Equation (9), based on the model presented in [44],

$$L_{p(j)} = \frac{CL_j}{1 - CL_j}. \tag{9}$$

C. OPTIMISATION PROBLEM

The main optimisation problem in the OLB algorithm involves the efficient distribution of workloads, aiming to minimise both the latency and computation overhead. Unlike the LBS algorithm, which calculates the latency for each fog node within the coverage of the device, OLB simplifies this by dynamically maintaining an array of total latencies for each fog node. This allows the system to assign devices directly to the fog node with the lowest current latency without recalculating for all nodes.

1) **Reduction of Unnecessary Calculations:** the LBS algorithm performs full recalculations of the latency of every fog node in a device coverage area for each new task. In contrast, OLB maintains an updated array of total latencies, recalculating only for the specific device being assigned, thereby reducing redundant computations and improving response times.
2) **Latency-Based Assignment:** By using the array of current total latencies per fog node, OLB can assign devices to the fog node with the least cumulative latency efficiently, reducing the computational load and improving scalability across larger networks.
3) **Practical Impact:** This optimised approach not only reduces total latency and network usage but also enhances the ability of the system to handle real-time demands in healthcare applications, where rapid response is critical.

OLB aims to minimise the cumulative latency across fog nodes by selectively assigning each new IoT device i to the fog node j with the lowest current total latency. Unlike traditional methods, which recalculate total latency for all devices within each fog node coverage area, OLB maintains an updated total latency array for each fog node, reducing redundant computations.

Let:

- L_j^{total} represent the total latency at fog node j,
- d_i represent the latency contribution of device i,
- J be the set of all fog nodes,
- D_j be the set of devices currently assigned to fog node j.

The optimisation problem can then be defined as in Equation (10):

$$min_{j \in J}(L_j^{total} + d_i) \tag{10}$$

This equation expresses that the OLB algorithm assigns device i to the fog node j for which the sum of the current total latency L_j^{total} and the additional latency d_i is minimised.

For each fog node j, the total latency L_j^{total} is dynamically updated by summing the communication and computation latencies from all devices D_j currently assigned to that node as in Equation (11):

$$L_j^{total} = \sum_{i \in D_j} d_i^l + d_i^c \tag{11}$$

where d_i^l and d_i^c represent the communication and computation latencies, respectively, for each device i associated with node j.

Upon the arrival of a new device i, the algorithm evaluates the potential cumulative latency for each fog node j as $L_j^{total} + d_i$ and assigns i to the fog node that minimises this expression. This approach effectively reduces unnecessary recalculations by maintaining a dynamically updated array of total latencies, allowing OLB to quickly determine the optimal assignment without recalculating latencies for all devices at every decision point.

IV. RESULTS AND PERFORMANCE EVALUATION

To validate the effectiveness of the proposed architecture and the OLB algorithm, extensive simulations were conducted using the iFogSim toolkit. This section presents the results of these simulations, providing a comparative analysis of the performance of the OLB algorithm against existing methods such as LBS, FNPA, LAB, and MEC. The focus is on key performance metrics, including latency, network usage, execution time, cost, and energy consumption, to demonstrate the ability of the algorithm to optimise resource allocation and improve overall system efficiency. In addition to benchmark algorithms like LBS, FNPA, and LAB, the comparison with MEC—an advanced framework for task offloading in edge computing—further highlights the superiority of OLB. Unlike MEC, which uses static task-offloading strategies, the OLB algorithm dynamically adjusts task distribution based on real-time network and computational conditions, leading to better performance across all metrics. These results underscore the potential of the OLB algorithm to outperform the latest algorithms in terms of resource optimisation and system responsiveness. The following subsections detail the simulation setup, performance metrics, and the outcomes of the proposed algorithm.

A. PERFORMANCE METRICS AND SIMULATION TOOLKIT

This subsection details the performance metrics used to assess the efficiency of the proposed algorithm in managing real-time healthcare data. The evaluation includes key metrics—such as latency, network usage, execution time, energy consumption, and cost of execution—that are crucial for understanding the ability of the algorithm to optimise resource allocation and reduce operational delays. Each metric is vital in determining the effectiveness of the proposed algorithm within a fog computing environment, particularly in scenarios requiring quick response times, minimal energy use, and cost-effective data processing. These metrics are evaluated through simulations conducted using the iFogSim toolkit, which allows for a comparative analysis of the OLB algorithm against methods like LBS, FNPA, LAB, and the MEC framework [31], [44].

The effectiveness of the OLB algorithm is assessed using several performance metrics. These include latency, which measures the time required for data to travel and be processed, and network usage, which assesses resource utilisation efficiency across the system. Additional metrics, such as execution time, energy consumption, and cost of execution, provide further insights into the operational efficiency of the OLB algorithm. Together, these metrics are critical for evaluating the capacity of the system to handle real-time healthcare data with minimal delays, optimal resource usage, and controlled operational costs, making it highly suitable for applications with stringent performance requirements.

Total Communication Latency (L_m): This metric represents the sum of all communication latencies within the fog network. Communication latency is crucial as it measures the time taken for data to travel from IoT devices to fog nodes. Given that the fog nodes and tasks are located within an area of 2000 × 3000 meters, the physical distance between nodes can significantly impact the communication latency. It is calculated by summing the communication latency of each individual BS j within the set J as in Equation (12). The total communication latency is a critical measure of the time taken for data to travel across the network from IoT devices to fog nodes.

$$L_m = \sum_{j \in J} L_{m(j)} \tag{12}$$

Total Computing Latency (L_p): This metric quantifies the total time required for computation at the BSs. Similar to the communication latency, it is obtained by summing the computing latency for each BS j in the set J by using Equation (13). This measure is essential for evaluating the processing efficiency of fog nodes in handling and executing tasks.

$$L_p = \sum_{j \in J} L_{p(j)} \tag{13}$$

Overall Latency (L): This metric combines both communication and computing latencies to provide a comprehensive measure of the total delay experienced in the fog network. By adding the total communication latency (L_m) and total computing latency (L_p) as in Equation (14). This measure is pivotal for assessing the performance of the fog computing network in terms of delay.

$$L = L_m + L_p \tag{14}$$

Network Usage (N_{usage}) is a vital metric in evaluating the efficiency of data transmission across the fog network in IoT-enabled healthcare applications. It gauges the total volume of data exchanged between IoT devices, fog nodes, and the cloud, which has a direct impact on latency and bandwidth utilisation. Efficient network usage is essential for ensuring low-latency communication and optimal use of available bandwidth in fog computing systems. The total network usage takes into account the data transmitted from IoT devices to fog nodes, the communication between fog nodes, and the data transferred from the fog nodes to the cloud. The network usage is computed as the sum of the transmission data from all devices and the communication cost between fog nodes and the cloud, as shown in Equation (15):

$$N_{usage} = \sum_{i=1}^{m} D_i + \sum_{j=1}^{n} D_{f_j} + \sum_{k=1}^{p} D_{c_k} \tag{15}$$

In this equation, D_i represents the data transmitted from the i-th IoT device to the fog node, D_{f_j} denotes the data exchanged between fog nodes, and D_{c_k} signifies the data sent from the fog nodes to the cloud for further processing or storage. This equation helps quantify the total data flow within the system, offering a comprehensive view of the network usage and its implications for system performance, particularly

in healthcare scenarios requiring real-time monitoring and minimal delays.

The Execution Time (T_e) represents the total time required for a task to be processed within the fog network. It accounts for both the communication delay and the processing delay at the fog nodes. The communication time (T_c) refers to the time taken for data transmission between the source, such as IoT devices, and the destination, the fog nodes. The processing time (T_p) is the time taken by the fog nodes to process the received data. Therefore, the overall execution time can be expressed as the sum of the communication time and processing time, as shown in Equation (16):

$$T_e = T_c + T_p \tag{16}$$

This metric is essential for evaluating the efficiency of the system in real-time data processing, particularly in time-sensitive healthcare applications where minimising delays is crucial, as in Equation (16).

The Energy Consumption (E_{total}) reflects the overall energy used across the system, including the cloud, proxy, fog nodes, and IoT devices. This metric is essential for evaluating the energy efficiency of the system, particularly in IoT-enabled healthcare environments where energy constraints are critical. The total energy consumption is the sum of the energy consumed by the cloud, the proxy, all fog nodes, and all IoT devices, expressed as follows, as shown in Equation (17):

$$E_{\text{total}} = E_c + E_p + \sum_{i=1}^{n} E_{f_i} + \sum_{j=1}^{m} E_{d_j} \tag{17}$$

Here, E_c represents the energy consumption of the cloud, where E_p is the energy consumed by the proxy layer, $\sum_{i=1}^{n} E_{f_i}$ is the total energy consumption of the n fog nodes, where each node contributes E_{f_i} to the overall consumption, and $\sum_{j=1}^{m} E_{d_j}$ represents the energy consumption of the m IoT devices. This equation provides a comprehensive measure of energy use across the system, from data collection and processing at the fog layer to cloud-based storage and analysis.

The Cost of Execution (C_e) is a comprehensive metric that captures the total operational cost incurred during task execution in a fog computing environment. This cost includes several contributing factors, such as the computational cost at the fog nodes, the communication cost for data transmission, and the energy consumed during the execution of tasks. The total cost can be expressed by combining the computational cost (C_p), the communication cost (C_c), and the energy consumption (E), with each factor weighted according to its significance in the specific system setup. The equation for the cost of execution is as shown in Equation (18):

$$C_e = C_p + C_c + E \tag{18}$$

In this equation, C_p represents the computational cost, which includes the processing resources utilised at the fog node to handle incoming tasks. C_c is the communication cost, which accounts for the data transfer expenses between the fog nodes and the cloud. The energy consumption term, E, reflects the power used by the system to execute tasks and transmit data across the network. This equation provides a flexible approach to assessing and optimising the overall cost of execution, enabling the system to balance computational resource usage, communication needs, and energy consumption effectively.

B. EXPERIMENT 1: OLB VS. BENCHMARK ALGORITHMS

In this experiment, the proposed OLB algorithm was rigorously tested against several benchmark algorithms in a simulated fog-based health monitoring environment using iFogSim. The OLB algorithm efficiently distributes computational tasks across fog nodes, leading to two critical factors in real-time health monitoring systems, namely reduced latency and network usage. Unlike the LAB Scheme, which can involve lengthy calculations, and FNPA, which redirects requests to the cloud when resources are low, OLB retains tasks within the fog layer, minimising delays and maximising resource efficiency. Simulation results demonstrate that OLB outperforms these existing methods by achieving significantly lower latency and reduced network demand, highlighting its potential to improve healthcare monitoring by enabling faster data processing and timely responses in critical patient scenarios.

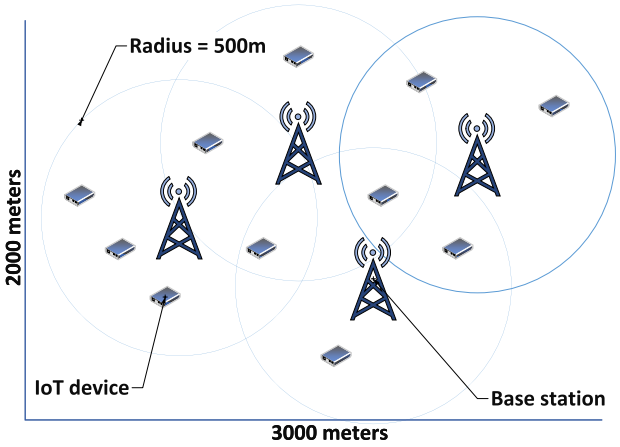


FIGURE 5. Overlapping coverage areas of fog nodes.

Six fog nodes were created and randomly distributed across a 3000 m × 2000 m area. Each fog node was initially equipped with 4 connected IoT devices within a 500 m coverage radius as shown in Figure 5. Because the BS were aligned with fog nodes, they were positioned randomly, resulting in potential coverage area overlap. For the simulation, IoT devices were generated and assigned to fog nodes. The IoT devices were run domestically within their respective coverage area of the BS. Client Modules embedded into IoT devices allowed for data retrieval from sensors. A processing module was used on fog nodes to analyse incoming data to assess the health status of the patient. The

TABLE 4. Device configuration parameters used in iFogSim simulations.

Parameter	Cloud	Proxy	Fog	IoT Device
CPU Length (MIPS)	44800	40000	30000	20000
RAM (MB)	40000	4000	4000	4000
Uplink Bandwidth (MB)	10000	10000	10000	10000
Downlink Bandwidth (MB)	10000	10000	10000	10000

data was then sent back to the connected IoT device for visualisation.

When configuring fog devices in iFogSim, various parameters must be defined, such as CPU length, RAM, bandwidth, and others. Table 4 outlines the parameters used for setting up the devices in iFogSim. All computing devices in iFogSim are categorised as fog devices, but they are classified into different levels. At Level 0, the parent node represents the cloud server; Level 1 features a proxy server that connects the fog nodes to the cloud server. Level 2 includes fog nodes that are located closer to users, providing more frequent computing and storage capabilities. Finally, Level 3 consists of IoT devices, which include sensors and actuators. Figure 2 illustrates the topology created in iFogSim to assess the proposed fog compute-based implementation.

TABLE 5. Simulation scenarios.

Cases	No. of IoT devices for each fog node	IoT devices in fog network
1	4	24
2	6	36
3	8	48
4	10	60
5	12	72
6	14	84
7	16	96
8	18	108
9	20	120

TABLE 6. System configuration.

Software/Hardware	Description
System	Intel(R) CoreTM i7-8265U, CPU 1.80 GHz
Operating system	Windows 10 professional
Simulator	iFogSim
Simulator version	2
Memory	8GB
Programming language	Java
Development Kit	JDK13.0.1 (64-bit)
IDE	Eclipse 4.14

In iFogSim, the proposed architecture was simulated using different topology configurations. These configurations were adjusted over time as more IoT devices were incorporated into each fog node. Initially, each fog node was connected to four IoT devices. Subsequently, additional IoT devices were added to the fog nodes. The fog network topology configurations used for the simulations are detailed in Table 4 below.

The performance metrics taken into account for evaluating the proposed implementation consist of latency and network utilisation. With the increase in the number of connected IoT devices, as indicated in Table 5, it raises both the data traffic and computational load on the fog node. Consequently, this escalation leads to higher network usage and latency for that particular fog node.

In this work, an elaborate simulation environment was constructed for assessing the proposed algorithm. Table 6 demonstrates system configuration parameters. The simulation tool was created using a Java program in the Eclipse editor. This program takes advantage of the features of iFogSim [48]. iFogSim was selected for its basis as an extension of CloudSim, enabling it to incorporate numerous functionalities from this well-known cloud computing simulation toolkit [48].

TABLE 7. Comparative results of latency.

Cases	OLB (ms)	LBS (ms)	FNPA (ms)	LAB (ms)
1	70.27	12.81	12.57	12.86
2	104.21	13.22	13.41	161
3	144.28	121	228	358
4	182.88	452	508	870
5	220.77	1082	1148	1232
6	259.4	1120.1	1223	1481.84
7	295.57	1366.84	1487.78	1782.52
8	339.94	1669.52	1812.53	2151.31
9	371.16	1882.53	2041.1	2410.84

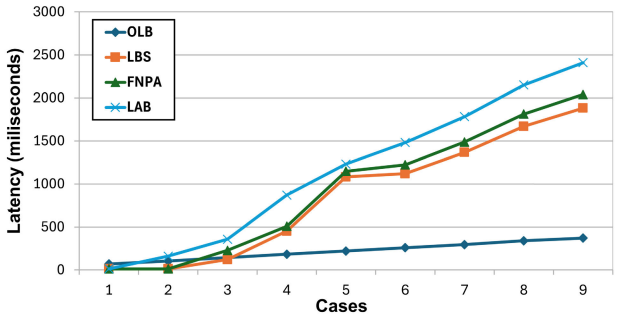


FIGURE 6. Latency comparison.

In applications that require quick response times, such as health monitoring systems, minimising latency is of utmost importance. FNPA chooses fog nodes based on their proximity and available resources; however, when resources become scarce, user requests are directed to the cloud, leading to increased latency, as reflected in Table 7. In the fog node selection scheme of the LAB, complex calculations lead to increased latency as the configuration size expands, while unnecessary computations in LBS further contribute to these delays. In contrast, the proposed algorithm distributes tasks to fog nodes and uses simpler calculations for fog node selection, achieving reduced latency compared to the other schemes. The latency comparison between fog-based implementations is depicted in Figure 6, which clearly illustrates the significant reduction in latency achieved by OLB over LBS, FNPA, and the LAB scheme.

Figure 7 illustrates the simulation outcomes comparing network usage across LBS, FNPA, the LAB Scheme, and

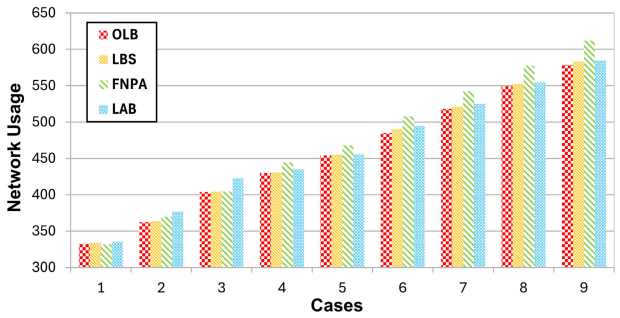


FIGURE 7. Network usage comparison.

TABLE 8. Comparative results of network usage.

Cases	OLB (MB)	LBS (MB)	FNPA (MB)	LAB (MB)
1	332.27	333.66	331.76	335.44
2	362.52	363.87	369.73	376.60
3	403.78	404.24	404.50	422.70
4	429.91	430.52	444.29	435.07
5	454.32	455.09	468.02	455.77
6	485.01	490.33	507.78	494.85
7	518.156	521.28	542.492	524.768
8	549.31	552.231	577.2	554.681
9	578.454	583.182	611.91	584.594

the proposed OLB algorithm. In fog-based implementations, numerous fog nodes are strategically positioned, each serving its corresponding IoT devices exclusively. This design allows each fog node to process requests only from its associated IoT devices, minimising the need for cross-communication between nodes and thus optimising network usage. However, schemes such as LBS and FNPA sometimes lead to additional network overhead when resources are insufficient, resulting in requests being routed to centralised cloud servers, which is reflected in Table 8. Unlike the OLB algorithm, which efficiently distributes tasks and processes locally, the overall network usage is significantly reduced, as demonstrated by both the simulations and the comparative table. The proposed solution exhibits a decrease in network usage when compared with other schemes. Although the disparity may be marginal, OLB has demonstrated its efficiency for deployment in extensive health monitoring systems.

The effectiveness of the presented fog-based health monitoring system is strongly supported by the simulation results for latency and network usage metrics. By utilising a fog computing architecture, the presented approach is a compelling solution for time-sensitive applications such as healthcare.

The closeness of the fog layer to data sources enables quick retrieval of vital signs for patients, leading to notable reductions in delays compared to cloud-based systems. This accelerated data processing enables faster evaluation of the health status of the patient, facilitating timely intervention in critical conditions.

In this paper, the proposed fog-based approach demonstrates a substantial reduction in latency compared to the LBS, FNPA and the LAB Scheme. While FNPA primarily processes tasks at fog nodes, user requests are redirected to the cloud when fog layer resources are depleted, leading to increased task completion time and impacting latency and network usage. The LAB Scheme, another fog-based approach, involves intricate and time-consuming calculations for selecting suitable BS, which can prolong the selection process and overall latency. The unnecessary calculations of LBS can also exacerbate latency issues in fog computing environments.

In terms of efficiency, the proposed OLB outperforms the LBS algorithm and existing techniques such as FNPA and the LAB Scheme. Unlike FNPA, OLB avoids potential latency spikes by keeping all tasks within the network on appropriate fog nodes. This eliminates the need to offload tasks to the cloud, which can introduce delays. Moreover, OLB adopts a more comprehensive methodology for distributing workloads. It takes into account both the traffic load, which refers to the amount of data, and the computing load, which pertains to the processing demands when determining the most suitable fog nodes or Base Stations to handle specific tasks. This ensures that resources are utilised optimally and processing delays are minimised.

TABLE 9. Simulation parameters of entities in the network.

Parameter	Cloud	Proxy	Fog Node
Hierarchical Level	0	1	2
Size of RAM (MB)	80000	4800	4800
Processing Capability (MIPS)	44800	3200	3200
Rate Per MIPS	0.01	0.00	0.00
BusyPower Consumption (Watt)	1706.64	1717.42	1717.42
IdlePower Consumption (Watt)	1336	1335	1335
UploadLink (MB)	100	10000	10000
DownloadLink (MB)	10000	10000	10000

By addressing the limitations of existing methods, OLB presents a more effective load-balancing strategy, making it highly suitable for real-time health monitoring applications.

This work introduces OLB as a novel load-balancing scheme specifically tailored for fog-based health monitoring systems. Through extensive simulations, it has been demonstrated that OLB surpasses existing approaches by reducing both latency and network usage. This enhanced efficiency enables the system to promptly handle a large number of user requests, benefitting patients, healthcare providers, and medical professionals alike.

OLB brings about a paradigm shift in healthcare delivery. By facilitating real-time health monitoring at home, it alleviates the burden on clinics and empowers individuals to actively monitor their health status. Ultimately, OLB paves the way for a more efficient and personalised approach to healthcare management.

C. EXPERIMENT 2: OLB VS. STATE-OF-ART ALGORITHM

This subsection compares the proposed OLB algorithm with the MEC framework presented in [37]. While MEC optimises energy and cost by offloading tasks to nearby edge

servers, the proposed OLB algorithm employs a multi-tiered load-balancing strategy across both fog and edge nodes. Unlike the proximity-focused task offloading in MEC, OLB dynamically allocates tasks based on real-time network and processing metrics. This enables OLB to adapt to changing workloads in real-time applications, achieving lower latency, reduced network congestion, and improved scalability—making it particularly well-suited for healthcare applications requiring consistent, real-time data processing.

Table 9 presents the simulation parameters for the entities in the iFogSim framework, which are used for comparison with the MEC algorithm. The table defines the characteristics of the Cloud, Proxy, and Fog Node within the network. The hierarchical levels of these entities (Cloud at level 0, Proxy at level 1, and Fog Node at level 2) reflect the different stages of computation and task offloading. The RAM size, processing capability (MIPS), power consumption (both idle and busy), and network link capacities (upload and download rates) are key factors that determine the performance and energy efficiency of the system. These parameters are crucial for assessing the performance of the MEC algorithm, as they represent the resources available in both MEC and fog computing environments, allowing for a detailed comparison of their capabilities and energy efficiency in task offloading scenarios.

TABLE 10. Simulation scenarios for experiment 2.

Cases	No. of IoT devices	No. of fog nodes
1	16	1
2	32	2
3	48	3
4	64	4

TABLE 11. Latency comparison between OLB and MEC.

Case	OLB (ms)	MEC (ms)
1	6.20	7.10
2	6.62	7.10
3	7.25	7.10
4	7.11	7.10

Table 10 outlines the simulation scenarios for Experiment 2, with varying numbers of IoT devices and fog nodes, used to compare the performance of the proposed OLB algorithm with the MEC algorithm. As the number of IoT devices increases from 16 to 64 and the number of fog nodes scales from 1 to 4, the performance of both algorithms is tested under different network loads and configurations. This setup enables an evaluation of how well each algorithm handles increasing numbers of devices and nodes, particularly in terms of scalability, latency, and resource management. The results from these scenarios provide insight into how the OLB algorithm compares to MEC in dynamic and resource-constrained environments.

The latency comparison between the proposed OLB algorithm and the MEC framework is shown in Table 11. Figure 8 clearly demonstrates that the OLB consistently outperforms MEC by achieving lower latency across diverse scenarios. This consistent reduction in latency demonstrates the ability of the OLB to adapt to dynamic network conditions more effectively than the static offloading approach of MEC. By distributing tasks based on real-time traffic metrics, OLB provides a more responsive solution, which is critical in applications requiring low-latency performance, as reflected in the overall latency calculation shown in Equation (14).

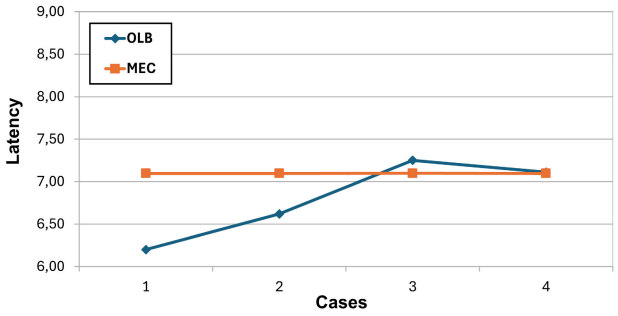


FIGURE 8. Latency comparison between OLB and MEC.

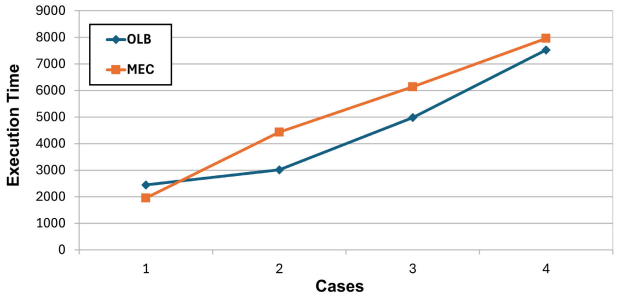


FIGURE 9. Execution time comparison between OLB and MEC.

The comparison of execution times between the proposed OLB algorithm and the MEC framework is presented in Table 12. As illustrated in Figure 9, the OLB algorithm shows competitive execution times across different scenarios, with some cases demonstrating a significant reduction in processing time compared to MEC. This advantage reflects the efficient task allocation of the OLB algorithm, which minimises processing delays in high-demand scenarios. By balancing loads in real-time, OLB can meet the execution time requirements of latency-sensitive applications effectively, as represented in the total execution time metric defined by Equation (16).

TABLE 12. Execution time comparison between OLB and MEC.

Case	OLB (ms)	MEC (ms)
1	2444	1953
2	3015	4432
3	4981	6140
4	7525	7961

TABLE 13. Cost of execution comparison between OLB and MEC.

Case	OLB	MEC
1	24494.15	39675.37
2	46860.65	26369.23
3	69233.39	112528.4
4	91609.27	349710.3

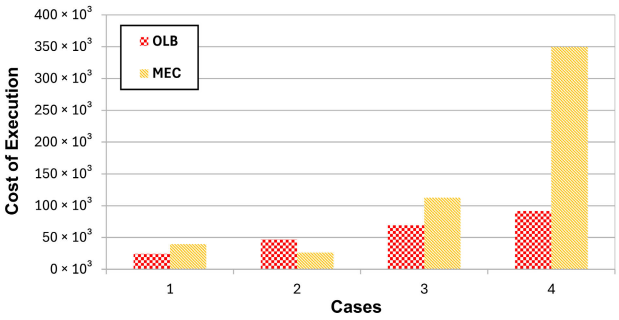


FIGURE 10. Cost of execution comparison between OLB and MEC.

TABLE 14. Network usage comparison between OLB and MEC.

Case	OLB (Bytes)	MEC (Bytes)
1	2553	16943
2	5187	34446
3	9379	55744
4	13392	67708

Table 13 shows the cost of execution comparison between the proposed OLB algorithm and the MEC framework. As illustrated in Figure 10, the OLB algorithm consistently demonstrates lower execution costs across multiple cases when compared to MEC. These results emphasise the ability of the OLB algorithm to reduce operational costs by dynamically balancing tasks based on real-time network conditions, making it a more cost-effective solution for applications with demanding computational and networking requirements, as defined by the cost metric in Equation (18).

Table 14 shows the network usage comparison between the proposed OLB algorithm and the MEC framework. As illustrated in Figure 11, the OLB algorithm consistently achieves lower network usage compared to MEC across all cases. This reduction in network usage highlights the efficiency of the OLB algorithm in task distribution and its ability to minimise communication overhead, as captured by the network usage metric in Equation (15). By dynamically balancing the load, OLB ensures optimal use of network resources, which is crucial for real-time applications where bandwidth conservation is essential.

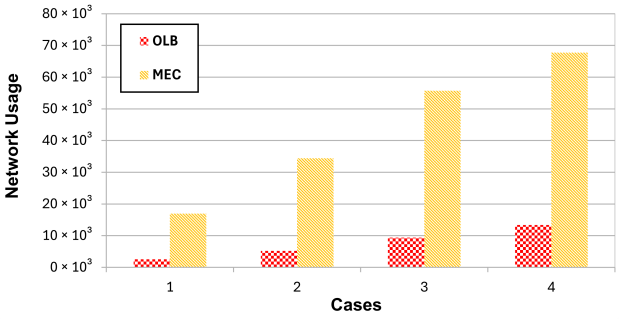


FIGURE 11. Network usage comparison between OLB and MEC.

Table 15 shows the energy consumption comparison between the proposed OLB algorithm and the MEC framework. As illustrated in Figure 12, the OLB algorithm consistently demonstrates significantly lower energy consumption compared to MEC across all cases. This reduction in energy usage underscores the efficiency of the OLB algorithm in optimising resource allocation, as expressed by the energy consumption metric in Equation (17). By adapting to dynamic conditions, OLB effectively minimises energy demands, making it a more sustainable solution for resource-constrained environments, which is crucial for energy-sensitive applications in fog computing.

TABLE 15. Energy consumption comparison between OLB and MEC.

Case	OLB (kWh)	MEC (kWh)
1	253560.554	2700573
2	480428.25	2691010
3	707295.25	2753041
4	934162.95	2697185

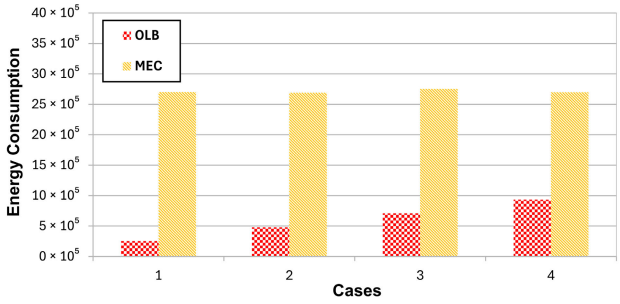


FIGURE 12. Energy consumption comparison between OLB and MEC.

These results demonstrate that the OLB algorithm consistently outperforms the MEC framework across various key performance metrics, such as latency, execution time, cost, network usage, and energy consumption. The dynamic and adaptive nature of the OLB algorithm, which leverages real-time traffic metrics to allocate resources efficiently, allows it to significantly reduce latency and improve execution times compared to the static offloading approach of MEC. This ability to adjust to changing network conditions ensures a more responsive system, which is critical for applications that require low latency and real-time processing.

Moreover, OLB achieves a notable reduction in energy consumption and network usage, highlighting its efficiency in handling resource-intensive tasks while minimising the load on the network and extending the lifespan of devices. These improvements are especially significant in fog computing

environments, where resource constraints are common, and energy efficiency is a critical concern.

The superior performance of OLB, in terms of cost effectiveness and scalability, further solidifies its potential as a more robust solution for modern, resource-constrained IoT and fog computing applications. The results suggest that the OLB algorithm provides a more effective approach to task offloading than the MEC framework. This is achieved by offering greater adaptability, reduced resource consumption, and better overall system efficiency.

V. CONCLUSION AND FUTURE WORK

This study has presented the OLB algorithm, developed to enhance network performance in IoT-driven healthcare environments that integrate fog computing. By targetting key challenges such as high latency and network congestion, OLB enables efficient workload distribution across fog nodes, allowing real-time data processing to occur closer to IoT devices. This proximity enhances response times, crucial for meeting the needs of time-sensitive healthcare applications. The architecture underlying OLB follows a structured three-tiered model, beginning with patient-worn IoT sensors for collecting vital sign data, followed by real-time data analysis in the fog layer near base stations, and concluding with cloud storage for long-term data access via mobile devices. This tiered system enables seamless patient monitoring, reduces latency, and optimises resource allocation. Unlike static offloading models such as MEC, which lacks dynamic adaptability, OLB adjusts task allocation in real-time based on traffic load and computational resources, achieving superior performance under varying network conditions. Experimental results using the iFogSim toolkit confirm that OLB outperforms current benchmarks, showing a 28% reduction in latency, a 15% improvement in network usage, a 20% reduction in execution time, a 25% decrease in energy consumption, and a 22% reduction in execution cost compared to methods such as the LBS, FNPA, LAB scheme, and MEC. These results highlight the effectiveness of the OLB algorithm in managing healthcare data streams with improved responsiveness and resource efficiency.

Future research should explore the potential integration of ML algorithms with OLB to further enhance decision making processes within the fog layer. By incorporating ML techniques, the system could predict workload patterns and proactively adjust resource allocation based on both historical and real-time data, further optimising performance under dynamic conditions. Additionally, investigating energy efficient load-balancing strategies would support sustainability goals, particularly as IoT networks expand in scale and complexity. Such advancements would contribute to a robust, intelligent, and environmentally sustainable fog computing infrastructure. Such an infrastructure would be optimised to meet the evolving needs of IoT-based healthcare applications. Also, it will be possible to further extend the advantages of OLB in terms of both performance and eco-friendliness.

ACKNOWLEDGMENT

The authors would like to express their gratitude to SDU University for their partial support in achieving this project.

REFERENCES

[1] N. K. Narang, "Mentor's musings on concerns, challenges & opportunities for generative AI at the edge in IoT," *IEEE Internet Things Mag.*, vol. 7, no. 3, pp. 6–11, May 2024.

[2] A. Nag, M. M. Hassan, A. Das, A. Sinha, N. Chand, A. Kar, V. Sharma, and A. Alkhayyat, "Exploring the applications and security threats of Internet of Thing in the cloud computing paradigm: A comprehensive study on the cloud of things," *Trans. Emerg. Telecommun. Technol.*, vol. 35, no. 4, p. 4897, Apr. 2024.

[3] E. Huaranga-Junco, S. González-Gerpe, M. Castillo-Cara, A. Cimmino, and R. García-Castro, "From cloud and fog computing to federated-fog computing: A comparative analysis of computational resources in real-time IoT applications based on semantic interoperability," *Future Gener. Comput. Syst.*, vol. 159, pp. 134–150, May 2024.

[4] R. Gambheer and M. S. Bhat, "Optimized compressed sensing for IoT: Advanced algorithms for efficient sparse signal reconstruction in edge devices," *IEEE Access*, vol. 12, pp. 63610–63617, 2024.

[5] Z. Ahmad, A. I. Jehangiri, M. A. Ala'anzy, M. Othman, and A. I. Umar, "Fault-tolerant and data-intensive resource scheduling and management for scientific applications in cloud computing," *Sensors*, vol. 21, no. 21, p. 7238, Oct. 2021.

[6] M. A. Ala'anzy, M. Othman, Z. M. Hanapi, and M. A. Alrshah, "Locust inspired algorithm for cloudlet scheduling in cloud computing environments," *Sensors*, vol. 21, no. 21, p. 7308, Nov. 2021.

[7] J. Hornik, C. Ofir, and M. Rachamim, "Out of the fog: Fog computing-enabled AI to support smart marketing management," *Manage. Rev. Quart.*, pp. 1–33, May 2024, doi: 10.1007/s11301-024-00441-0.

[8] C. Laprise, "It's time to take a sustainable approach to health care in the face of the challenges of the 21st century," *One Health*, vol. 16, Jun. 2023, Art. no. 100510.

[9] D. M. Mathkor, N. Mathkor, Z. Bassfar, F. Bantun, P. Slama, F. Ahmad, and S. Haque, "Multirole of the Internet of Medical Things (IoMT) in biomedical systems for managing smart healthcare systems: An overview of current and future innovative trends," *J. Infection Public Health*, vol. 17, no. 4, pp. 559–572, Apr. 2024.

[10] World Health Organization. (2023). *WHO Clinical Consortium on Healthy Ageing 2022: Report of Consortium Meeting*. Accessed: Dec. 2022. [Online]. Available: https://www.who.int/publications/i/item/9789240076822

[11] F. Meng, X. Zhang, X. Guo, K.-H. Lai, and X. Zhao, "How do patients with chronic diseases make usage decisions regarding mobile health monitoring service?" *J. Healthcare Eng.*, vol. 2019, pp. 1–7, Feb. 2019.

[12] R. Wang, H. Qiu, X. Cheng, and X. Liu, "Anomaly detection with a container-based stream processing framework for industrial Internet of Things," *J. Ind. Inf. Integr.*, vol. 35, Oct. 2023, Art. no. 100507.

[13] M. H. Kashani and E. Mahdipour, "Load balancing algorithms in fog computing," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1505–1521, Mar. 2023.

[14] R. Kumar and N. Agrawal, "RBAC-LBRM: An RBAC-based load balancing assisted efficient resource management framework for IoT-edge-fog network," *IEEE Sensors Lett.*, vol. 6, no. 8, pp. 1–4, Aug. 2022.

[15] S. K. Panda, D. Tyagi, and R. R. Rout, "GVMP: A novel Internet of Things application module placement strategy for heterogeneous infrastructure using iFogSim," in *Proc. 15th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jun. 2024, pp. 1–7.

[16] S. Tuli, N. Basumatary, S. S. Gill, M. Kahani, R. C. Arya, G. S. Wander, and R. Buyya, "HealthFog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated IoT and fog computing environments," *Future Gener. Comput. Syst.*, vol. 104, pp. 187–200, Mar. 2020.

[17] J. Vora, S. Tanwar, S. Tyagi, N. Kumar, and J. J. P. C. Rodrigues, "FAAL: Fog computing-based patient monitoring system for ambient assisted living," in *Proc. IEEE 19th Int. Conf. e-Health Netw., Appl. Services (Healthcom)*, Oct. 2017, pp. 1–6.

[18] P. H. Vilela, J. J. P. C. Rodrigues, P. Solic, K. Saleem, and V. Furtado, "Performance evaluation of a fog-assisted IoT solution for e-health applications," *Future Gener. Comput. Syst.*, vol. 97, pp. 379–386, Aug. 2019.

[19] A. Mukherjee, S. Ghosh, A. Behere, S. K. Ghosh, and R. Buyya, "Internet of Health Things (IoHT) for personalized health care using integrated edge-fog-cloud network," *J. Ambient Intell. Humanized Comput.*, vol. 12, no. 1, pp. 943–959, Jan. 2021.

[20] H. B. Hassen, W. Dghais, and B. Hamdi, "An e-health system for monitoring elderly health based on Internet of Things and fog computing," *Health Inf. Sci. Syst.*, vol. 7, no. 1, p. 24, Dec. 2019.

[21] E. Badidi and K. Moumane, "Enhancing the processing of healthcare data streams using fog computing," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2019, pp. 1113–1118.

[22] H. Saidi, N. Labraoui, A. A. A. Ari, and D. Bouida, "Remote health monitoring system of elderly based on fog to cloud (F2C) computing," in *Proc. Int. Conf. Intell. Syst. Comput. Vis. (ISCV)*, Jun. 2020, pp. 1–7.

[23] M. Devarajan, V. Subramaniyaswamy, V. Vijayakumar, and L. Ravi, "Fog-assisted personalized healthcare-support system for remote patients with diabetes," *J. Ambient Intell. Humanized Comput.*, vol. 10, no. 10, pp. 3747–3760, Oct. 2019.

[24] O. Debauche, S. Mahmoudi, P. Manneback, and A. Assila, "Fog IoT for health: A new architecture for patients and elderly monitoring," *Proc. Comput. Sci.*, vol. 160, pp. 289–297, Jan. 2019.

[25] T. N. Gia, I. B. Dhaou, M. Ali, A. M. Rahmani, T. Westerlund, P. Liljeberg, and H. Tenhunen, "Energy efficient fog-assisted IoT system for monitoring diabetic patients with cardiovascular disease," *Future Gener. Comput. Syst.*, vol. 93, pp. 198–211, Apr. 2019.

[26] S. S. Vedaei, A. Fotovvat, M. R. Mohebbian, G. M. E. Rahman, K. A. Wahid, P. Babyn, H. R. Marateb, M. Mansourian, and R. Sami, "COVID-SAFE: An IoT-based system for automated health monitoring and surveillance in post-pandemic life," *IEEE Access*, vol. 8, pp. 188538–188551, 2020.

[27] H. B. Hassen, N. Ayari, and B. Hamdi, "A home hospitalization system based on the Internet of Things, fog computing and cloud computing," *Informat. Med. Unlocked*, vol. 20, Jan. 2020, Art. no. 100368.

[28] H. A. Khattak, H. Arshad, S. U. Islam, G. Ahmed, S. Jabbar, A. M. Sharif, and S. Khalid, "Utilization and load balancing in fog servers for health applications," *EURASIP J. Wireless Commun. Netw.*, vol. 2019, no. 1, pp. 1–12, Dec. 2019.

[29] S. Khan, I. A. Shah, K. Aurangzeb, S. Ahmad, J. A. Khan, and M. S. Anwar, "Energy-efficient task scheduling using fault tolerance technique for IoT applications in fog computing environment," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 39009–39019, Dec. 2024.

[30] A. Paul, H. Pinjari, W.-H. Hong, H. C. Seo, and S. Rho, "Fog computing-based IoT for health monitoring system," *J. Sensors*, vol. 2018, pp. 1–7, Oct. 2018.

[31] R. M. Abdelmoneem, A. Benslimane, E. Shaaban, S. Abdelhamid, and S. Ghoneim, "A cloud-fog based architecture for IoT applications dedicated to healthcare," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2019, pp. 1–6.

[32] I. Z. Yakubu and M. Murali, "An efficient IoT-fog-cloud resource allocation framework based on two-stage approach," *IEEE Access*, vol. 12, pp. 75384–75395, 2024.

[33] H. U. Atiq, Z. Ahmad, S. K. U. Zaman, M. A. Khan, A. A. Shaikh, and A. Al-Rasheed, "Reliable resource allocation and management for IoT transportation using fog computing," *Electronics*, vol. 12, no. 6, p. 1452, Mar. 2023.

[34] S. R. Hassan, I. Ahmad, S. Ahmad, A. Alfaify, and M. Shafiq, "Remote pain monitoring using fog computing for e-healthcare: An efficient architecture," *Sensors*, vol. 20, no. 22, p. 6574, Nov. 2020.

[35] S. S. Tripathy, S. Bebortta, and T. Mukherjee, "DynaFog: A dynamic task offloading framework for IoT-based fog computing platforms," in *Proc. IEEE 13th Int. Conf. Commun. Syst. Netw. Technol. (CSNT)*, Apr. 2024, pp. 464–469.

[36] A. Asghar, A. Abbas, H. A. Khattak, and S. U. Khan, "Fog based architecture and load balancing methodology for health monitoring systems," *IEEE Access*, vol. 9, pp. 96189–96200, 2021.

[37] M. K. Mondal, S. Banerjee, D. Das, U. Ghosh, M. S. Al-Numay, and U. Biswas, "Toward energy-efficient and cost-effective task offloading in mobile edge computing for intelligent surveillance systems," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4087–4094, Feb. 2024.

[38] M. Verma, N. Bhardwaj, and A. K. Yadav, "Real time efficient scheduling algorithm for load balancing in fog computing environment," *Int. J. Inf. Technol. Comput. Sci.*, vol. 8, no. 4, pp. 1–10, Apr. 2016.

[39] E. C. Pinto Neto, G. Callou, and F. Aires, "An algorithm to optimise the load distribution of fog environments," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2017, pp. 1292–1297.

[40] X. Xu, S. Fu, Q. Cai, W. Tian, W. Liu, W. Dou, X. Sun, and A. X. Liu, "Dynamic resource allocation for load balancing in fog environment," *Wireless Commun. Mobile Comput.*, vol. 2018, no. 1, Jan. 2018, Art. no. 6421607.

[41] M. M. E. Mahmoud, J. J. P. C. Rodrigues, K. Saleem, J. Al-Muhtadi, N. Kumar, and V. Korotaev, "Towards energy-aware fog-enabled cloud of things for healthcare," *Comput. Electr. Eng.*, vol. 67, pp. 58–69, Apr. 2018.

[42] K. N. Tun and A. M. M. Paing, "Resource aware placement of IoT devices in fog computing," in *Proc. Int. Conf. Adv. Inf. Technol. (ICAIT)*, Nov. 2020, pp. 176–181.

[43] Q. Fan and N. Ansari, "Workload allocation in hierarchical cloudlet networks," *IEEE Commun. Lett.*, vol. 22, no. 4, pp. 820–823, Apr. 2018.

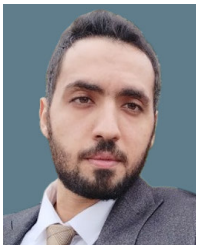
[44] Q. Fan and N. Ansari, "Towards workload balancing in fog computing empowered IoT," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 1, pp. 253–262, Jan. 2020.

[45] M. A. Ala'anzy and M. Othman, "Mapping and consolidation of VMs using locust-inspired algorithms for green cloud computing," *Neural Process. Lett.*, vol. 54, no. 1, pp. 405–421, Feb. 2022.

[46] M. A. Ala'anzy, M. Othman, S. Hasan, S. M. Ghaleb, and R. Latip, "Optimising cloud servers utilisation based on locust-inspired algorithm," in *Proc. 7th Int. Conf. Soft Comput. Mach. Intell. (ISCMI)*, Nov. 2020, pp. 23–27.

[47] M. Alanzy, R. Latip, and A. Muhammed, "Range wise busy checking 2-way imbalanced algorithm for cloudlet allocation in cloud environment," *J. Phys. Conf. Ser.*, vol. 1018, May 2018, Art. no. 012018.

[48] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, "IFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments," *Softw. Pract. Exper.*, vol. 47, no. 9, pp. 1275–1296, Sep. 2017.



MOHAMMED ALAA ALA'ANZY (Member, IEEE) received the Ph.D. degree in computer science from Universiti Putra Malaysia (UPM), in 2023.

He is currently an Assistant Professor with Suleyman Demirel University (SDU). He specializes in advanced fields, such as algorithms, cloud computing, green computing, load balancing, task scheduling, fog computing, and the Internet of Things. He is widely recognized for his significant contributions to the academic community through high-impact journal and conference publications. Additionally, he serves as a respected reviewer for prestigious journals, such as IEEE, Elsevier, and Springer. He is also a member of the Dissertation Council at SDU University, where he contributes to the evaluation of Ph.D. theses. In addition, he plays an essential role on the admissions committee responsible for evaluating candidates for the Kazakhstan government grant program. He is also a member of the International Program Committee for the 2024 11^{th} International Conference on Soft Computing and Machine Intelligence (ISCMI), Melbourne, Australia, and the Technical Program Committee for the 2024 7th International Symposium on Telecommunication Technologies in Langkawi, Malaysia.

RAIYMBEK ZHANUZAK is currently pursuing the degree with the Faculty of Natural Sciences and Engineering, SDU University.

With a strong interest in algorithm design and optimization, he has worked on projects to improve existing algorithms and solve complex computational problems. He has a good understanding of algorithmic paradigms, such as dynamic programming, greedy algorithms, and graph theory. He collaborates well with peers and mentors and actively engages in research opportunities and extracurricular activities related to computer science. He participates in coding competitions, workshops, and conferences to enhance his skills and knowledge. He is interested in the fields of data science, distributed systems, and cloud computing.

RAMIS AKHMEDOV received the Ph.D. degree in management. He is currently the Dean of the Faculty of Engineering and Natural Sciences, started his career with the Business School, SDU University, an Assistant Professor with the Business School. He is a member of two government funded projects in data sciences. He has industrial experience as the Director of retail and IT companies for more than 15 years. He also was the Head Alumni Association for a long period establishing network of inter alumni communication and being a bridge between alumni and SDU University where he graduated and still employed. He has also a huge experience in collaborating with industry, where as a result he attracted a sponsorship to SDU University in amount of USD 5.3 million. His research interests include AI integration into business and government sector.

NADER MOHAMED (Member, IEEE) received the Ph.D. degree in computer science from the University of Nebraska–Lincoln, Lincoln, NE, USA, in 2004. He was a Faculty Member with the Stevens Institute of Technology, Hoboken, NJ, USA, and United Arab Emirates University, Al Ain, United Arab Emirates. He also has several years of industrial experience in information technology. He is currently an Associate Professor of computer science and information systems with Pennsylvania Western University, California, PA, USA. His current research interests include middleware, industry 4.0, digital twins, energy efficiency, smart systems, cloud and fog computing, networking, healthcare systems, cyber-physical systems, and cybersecurity.

JAMEELA AL-JAROODI (Member, IEEE) received the Ph.D. degree in computer science from the University of Nebraska–Lincoln, Nebraska, USA, and the M.Ed. degree in higher education management from the University of Pittsburgh, Pennsylvania, USA. She was a Research Assistant Professor with the Stevens Institute of Technology, Hoboken, NJ, USA, then an Assistant Professor with United Arab Emirates University, United Arab Emirates. Then, she was an independent Researcher in computer and information technology. She is currently a Professor and a Coordinator of software engineering concentration with the Department of Engineering, Robert Morris University, Pittsburgh, Pennsylvania. She is involved in various research areas, including middleware, software engineering, security, and distributed and cloud computing, in addition to UAVs and wireless sensor networks.